算法与动态规划
中文复习手册

从题意到状态转移，再到可运行的 C++14 代码

适合初学者 · 14 个主题 · 反例图解 · 纠错与自检

> 阅读约定：原对话可核对的详细题面包括最大正方形、最小花费爬楼梯及跳跃最大得分的部分讨论。其余题目依照题名写成明确、可独立练习的版本；若比赛原题的输入与规则不同，以原题为准。所有样例均按本手册所写规则重新计算。

阅读路线：先看第 1 章的状态定义，再依次练线性 DP、二维 DP、背包、模拟、区间 DP、贪心和图论。每道题先遮住代码，根据“最后一步从哪里来”自己写转移。

目录：1 动态规划入门；2 最大正方形；3 最小花费爬楼梯；4 打家劫舍；5 弹簧板；6 跳跃最大得分；7 最大子段和；8 充电桩申请；9 机器人指令模拟；10 学生最高分；11 切水果；12 铺路；13 Dijkstra；14 猴子桃子；附录 纸牌均分与检查清单。

版本：2026 年 10 月 15 日

1 动态规划先问四件事

① 状态：dp[i] 到底表示“恰好到 i”，还是“前 i 个中的最优”？② 边界：第一个合法状态是什么？③ 转移：最后一次选择是什么？④ 顺序：计算当前状态时，依赖值是否已算好？

> 答案位置由状态定义决定：若 dp[i] 是“到达 i 的最小花费”，答案通常是 dp[n]；若 dp[i] 是“从 i 出发到终点的最少步数”，答案通常是 dp[0]。不要只凭模板猜下标。

何时倒序？若 dp[i] 依赖更大的下标（例如弹簧板跳到 i+a[i]），从右往左算。0/1 背包若在同一数组中原地更新，也必须让容量从大到小，以免一件物品被重复选中。

负数与不可达：求最大值时若答案可能为负，不能把 dp 默认设为 0；可用很小的哨兵值。加法前要检查前驱是否可达。

最小样例检查：空输入是否允许？n=1 能否输出？全负数会怎样？一维/二维数组首尾是否越界？

2 最大正方形：上、左、左上缺一不可

题意与样例

给定仅含 0/1 的 n×m 矩阵，求全为 1 的最大正方形边长。

样例：3×3：0 1 1 / 1 1 1 / 1 1 1；答案 2。

分析与状态

固定右下角 (i,j)。若该格为 1，要扩成更大的正方形，上方、左方、左上方三个重叠小正方形都得足够大。

dp[i][j] = 以 (i,j) 为右下角的最大全 1 正方形边长。

边界、转移与计算顺序

当前格为 0 时 dp=0；为 1 时 dp[i][j]=1+min(上,左,左上)。用额外第 0 行/列当 0 边界，按行从上到下、每行从左到右计算；答案是所有 dp 的最大值。

复杂度与易错点

复杂度：时间 O(nm)，空间 O(nm)。 常见错误：把最大边长误写成面积；漏掉左上；行列下标混用。

三张反例图：被省略的邻居恰好藏着一个 0

少左上

0	1	1
1	1	1
1	1	1

上 2 / 左 2 / 左上 1

右下角真实值 = 2

少上

1	1	0
1	1	1
1	1	1

上 1 / 左 2 / 左上 2

右下角真实值 = 2

少左

1	1	1
1	1	1
0	1	1

上 2 / 左 1 / 左上 2

右下角真实值 = 2

图中右下角都为 1；若去掉标题所示的邻居，另外两个 dp 都等于 2，会误算出边长 3。正确答案均为 2。更大正方形要覆盖右上、左下、左上三个角，因此三个约束缺一不可。

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n, m; cin >> n >> m;
    vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
    int ans = 0;
    for (int i=1; i<=n; ++i) for (int j=1; j<=m; ++j) {
        int x; cin >> x;
        if (x == 1) dp[i][j] = 1 + min({dp[i-1][j],
                                        dp[i][j-1], dp[i-1][j-1]});
        ans = max(ans, dp[i][j]);
    }
    cout << ans << '\n';
}
```

3 最小花费爬楼梯

题意与样例

第 0 至 n−1 阶各有费用 cost[i]；可从第 0 或第 1 阶免费起步。离开某阶时付费，可上 1 或 2 阶，求到顶部第 n 阶的最小费用。

样例：n=3，费用 10 15 20；从第 1 阶付 15 跳到顶部，答案 15。

分析与状态

到达 i 只能从 i−1 或 i−2 来；费用在“离开前一阶”时支付。

dp[i] = 到达第 i 阶所付的最小费用；顶部 n 没有 cost[n]。

边界、转移与计算顺序

dp[0]=dp[1]=0；i 从 2 到 n：dp[i]=min(dp[i−1]+cost[i−1], dp[i−2]+cost[i−2])；输出 dp[n]。

复杂度与易错点

复杂度：时间 O(n)，空间 O(n)。 常见错误：若 cost 从 1 开始输入，公式中的 cost[0] 错位；把顶部当成需要付费的台阶。



从 1 付 15，可直接到顶部 3

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> cost(n), dp(n+1, 0);
    for (auto &x : cost) cin >> x;
    for (int i=2; i<=n; ++i)
        dp[i] = min(dp[i-1]+cost[i-1],
                    dp[i-2]+cost[i-2]);
    cout << dp[n] << '\n';
}
```

4 打家劫舍

题意与样例

n 间房子排成一行，不能偷相邻两间，求最多金额；允许一间都不偷。

样例：2 7 9 3 1 → 12。

分析与状态

看第 i 间：不偷则保持前 i−1 间的最优；偷则加上前 i−2 间的最优。

dp[i] = 前 i 间房能取得的最大金额。

边界、转移与计算顺序

dp[0]=0，dp[1]=max(0,a[1])；i 从 2 到 n：dp[i]=max(dp[i−1],dp[i−2]+a[i])。

复杂度与易错点

复杂度：时间 O(n)，空间 O(n)。 常见错误：把 dp[i] 误当成“必须偷第 i 间”；忘记不偷选项。



选第 1、3、5 间：2+9+1=12

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> a(n+1), dp(n+1, 0);
    for (int i=1; i<=n; ++i) cin >> a[i];
    for (int i=1; i<=n; ++i)
        dp[i] = max(dp[i-1], (i>=2 ? dp[i-2] : 0LL)+a[i]);
    cout << dp[n] << '\n';
}
```

5 弹簧板：为什么从右往左算

题意与样例

教学版：站在位置 0；位置 i 的弹簧把人送到 i+a[i]，若位置 ≥n 就离开；求离开所需弹跳次数。保证 a[i]≥1。

样例：n=5，a=2 1 3 1 1：0→2→5，答案 2。

分析与状态

从 i 只会去更大的位置，因此后继的答案先算出来，再回填当前状态。

dp[i] = 从位置 i 出发离开数组的最少弹跳次数；i≥n 视作 0 次。

边界、转移与计算顺序

i 从 n−1 降到 0：若 i+a[i]≥n，则 dp[i]=1；否则 dp[i]=1+dp[i+a[i]]。输出 dp[0]。

复杂度与易错点

复杂度：时间 O(n)，空间 O(n)。 常见错误：正序计算尚未得到的后继；把落在 n 恰好出界误判为仍在数组。

计算方向图：0 → 2 → 5(出界)。填表时先算 4、3、2、1、0；答案存于起点 0。

C++14 代码

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n; cin >> n;
    vector<int> a(n), dp(n, 0);
    for (int &x : a) cin >> x;
    for (int i=n-1; i>=0; --i) {
        int nxt = i + a[i];
        dp[i] = 1 + (nxt >= n ? 0 : dp[nxt]);
    }
    cout << (n ? dp[0] : 0) << '\n';
}
```

6 跳跃最大得分

题意与样例

从第 1 个平台出发，起点分数计入。每次向右跳 1 至 k 格；到达平台 i 获得 v[i]，必须落到 n，求最大总分。

样例：n=5,k=2，v=1 −2 3 −1 4；选 1→3→5，答案 8。

分析与状态

计算到 i 时，枚举前面最多 k 个可跳来的平台，取其中最高总分，再加 v[i]。

dp[i] = 从 1 到达 i 的最大总分。

边界、转移与计算顺序

dp[1]=v[1]；i 从 2 到 n，dp[i]=v[i]+max(dp[j])，max(1,i−k)≤j<i。

复杂度与易错点

复杂度：时间 O(nk)，空间 O(n)。 常见错误：所有得分可能为负，dp 不能默认 0；起点 v[1] 不可漏计。

例如 i=5、k=2，只比较 j=3,4，即 max(dp[3],dp[4])+v[5]。数学记号 max 下方的范围就是内层 for 循环的取值范围。

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <limits>
using namespace std;
int main() {
    int n,k; cin >> n >> k;
    vector<long long> v(n+1), dp(n+1);
    for (int i=1; i<=n; ++i) cin >> v[i];
    dp[1]=v[1];
    for (int i=2; i<=n; ++i) {
        long long best=numeric_limits<long long>::lowest();
        for (int j=max(1,i-k); j<i; ++j) best=max(best,dp[j]);
        dp[i]=best+v[i];
    }
    cout << dp[n] << '\n';
}
```

7 最大子段和

题意与样例

从非空整数数组中选一个连续子段，求其元素和最大值。

样例：−2 1 −3 4 −1 2 1 −5 4 → 6（4,−1,2,1）。

分析与状态

以 i 结尾的最优子段，要么单独从 i 开始，要么接在以 i−1 结尾的最优子段后。

dp[i] = 必须以 i 结尾的非空子段最大和。

边界、转移与计算顺序

dp[1]=a[1]；i 从 2 到 n：dp[i]=max(a[i],dp[i−1]+a[i])；答案是 max_i dp[i]。

复杂度与易错点

复杂度：时间 O(n)，空间 O(n)，可压到 O(1)。 常见错误：全负数时答案仍是最大那个负数，不能初始化为 0；答案不一定是 dp[n]。

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n; cin >> n;
    vector<long long> a(n+1), dp(n+1);
    for (int i=1; i<=n; ++i) cin >> a[i];
    dp[1]=a[1]; long long ans=dp[1];
    for (int i=2; i<=n; ++i) {
        dp[i]=max(a[i],dp[i-1]+a[i]);
        ans=max(ans,dp[i]);
    }
    cout << ans << '\n';
}
```

8 充电桩申请：0/1 背包建模

题意与样例

教学版：共有 n 份申请，每份占 w[i] 个车位时段、带来收益 v[i]；容量 C，同一申请最多接一次，求最大收益。

样例：C=5；(w,v)=(2,6),(3,7),(4,9)；接前两份，答案 13。

分析与状态

每份申请只能选或不选。设容量 j 的最大收益，一维原地更新时要从大容量到小容量。

dp[j] = 已处理的申请中，在容量不超过 j 时的最大收益。

边界、转移与计算顺序

初始全部为 0；对每份申请，从 j=C 降到 w[i]，dp[j]=max(dp[j],dp[j−w[i]]+v[i])。输出 dp[C]。

复杂度与易错点

复杂度：时间 O(nC)，空间 O(C)。
常见错误：容量正序会让同一申请被重复选，变成完全背包；要看清“恰好”与“不超过”。

倒序图：处理 w=2,v=6 时，先更新 dp[5] 再 dp[4]、dp[3]、dp[2]。读取的 dp[j−2] 仍属于上一轮，故此申请只用一次。

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int main() {
    int n,C; cin >> n >> C;
    vector<long long> dp(C+1,0);
    for (int i=0; i<n; ++i) {
        int w; long long v; cin >> w >> v;
        for (int j=C; j>=w; --j)
            dp[j]=max(dp[j],dp[j-w]+v);
    }
    cout << dp[C] << '\n';
}
```

9 机器人指令模拟

题意与样例

教学版：n×m 地图中 . 可走、# 是墙。机器人从 (r,c) 出发，朝 N/E/S/W；依次执行 L 左转、R 右转、F 前进一步。遇到边界或墙时留在原地，输出最终位置和方向。

样例：3×3 全为点；起点 (2,2,N)，指令 F R F → (1,3,E)。

分析与状态

状态只有位置、方向。方向按 N,E,S,W 存成 0,1,2,3；左转减 1，右转加 1；前进时先试探再提交。

本题为模拟而非 DP；状态是 (r,c,d)。

边界、转移与计算顺序

每条指令顺序执行；F 时检查 1≤nr≤n、1≤nc≤m 且地图为 .。

复杂度与易错点

复杂度：时间 O(nm+|s|)，空间 O(nm)。 常见错误：题面坐标与数组下标差 1；撞墙后仍移动；左转方向更新符号写反。

C++14 代码

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
int main() {
    int n,m; cin >> n >> m;
    vector<string> g(n);
    for (auto &row:g) cin >> row;
    int r,c; char facing; string s;
    cin >> r >> c >> facing >> s;
    string dirs="NESW"; int d=dirs.find(facing);
    int dr[4]={-1,0,1,0}, dc[4]={0,1,0,-1};
    for (char op:s) {
        if (op=='L') d=(d+3)%4;
        else if (op=='R') d=(d+1)%4;
        else if (op=='F') {
            int nr=r+dr[d], nc=c+dc[d];
            if (nr>=1 && nr<=n && nc>=1 && nc<=m &&
                g[nr-1][nc-1]=='.') r=nr,c=nc;
        }
    }
    cout << r << ' ' << c << ' ' << dirs[d] << '\n';
}
```

10 学生最高分：区间查询与修改

题意与样例

教学版：n 位学生有分数；执行 q 次操作：Q l r 查询区间最高分，U p x 将第 p 人分数改为 x。

样例：分数 80 90 70；Q 1 3→90；U 2 60；Q 1 3→80。

分析与状态

区间最大值有“合并”性质：大区间最大值等于左右子区间最大值。线段树让单点修改、区间查询都只访问 O(log n) 个节点。

tree[o] = 节点 o 对应区间的最高分；叶子是单个学生。

边界、转移与计算顺序

建树时向上取 max；U 修改叶子后向上更新；Q 若区间完全覆盖直接返回，否则查询相交子区间并取 max。

复杂度与易错点

复杂度：建树 O(n)，每操作 O(log n)，空间 O(n)。 常见错误：数组开太小（通常 4n）；查询越界；更新后忘记向上维护。

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <limits>
using namespace std;
vector<int> a,t;
void build(int o,int l,int r){
    if(l==r){t[o]=a[l];return;}
    int m=(l+r)/2; build(o*2,l,m); build(o*2+1,m+1,r);
    t[o]=max(t[o*2],t[o*2+1]);
}
void upd(int o,int l,int r,int p,int x){
    if(l==r){t[o]=x;return;}
    int m=(l+r)/2;
    if(p<=m) upd(o*2,l,m,p,x);
    else upd(o*2+1,m+1,r,p,x);
    t[o]=max(t[o*2],t[o*2+1]);
}
int qry(int o,int l,int r,int ql,int qr){
    if(ql<=l && r<=qr) return t[o];
    int m=(l+r)/2, ans=numeric_limits<int>::min();
    if(ql<=m) ans=max(ans,qry(o*2,l,m,ql,qr));
    if(qr>m) ans=max(ans,qry(o*2+1,m+1,r,ql,qr));
    return ans;
}
int main(){
    int n,q;cin>>n>>q;a.resize(n+1);t.resize(4*n+4);
    for(int i=1;i<=n;++i)cin>>a[i];build(1,1,n);
    while(q--){char op;int x,y;cin>>op>>x>>y;
        if(op=='U')upd(1,1,n,x,y);
        else cout<<qry(1,1,n,x,y)<<'\n';
    }
}
```

11 切水果：区间 DP 的明确版本

题意与样例

教学版：一根水果条长 L，必须在给定的 n 个不同位置切开。每次切一段的代价等于该段当前长度；求总最小代价。

样例：L=10，切点 4、7；先切 4 再切 7：10+6=16；反过来为 10+7=17。

分析与状态

第一次在区间 (l,r) 的某个切点 k 切开，先付 r−l，左右两段再分别最优处理。

dp[i][j] = 在切点坐标 p[i] 与 p[j] 之间完成所有内部切割的最小代价。

边界、转移与计算顺序

加 p[0]=0,p[n+1]=L；相邻坐标 dp[i][i+1]=0。按区间跨度从小到大，dp[i][j]=p[j]−p[i]+min_{i<k<j}(dp[i][k]+dp[k][j])。

复杂度与易错点

复杂度：时间 $O(n^3)$，空间 $O(n^2)$。
常见错误：先按位置排序；不要漏掉左右端点；本版“切水果”是切割代价模型，不等同于其他同名题。

划分图：0 ── 4 ── 7 ── 10。第一次选 4，代价 10；右段 4..10 再切 7，代价 6。

C++14 代码

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
using namespace std;
int main(){
    int L,n;cin>>L>>n;
    vector<int> p(n+2);p[0]=0;p[n+1]=L;
    for(int i=1;i<=n;++i)cin>>p[i];
    sort(p.begin()+1,p.begin()+n+1);
    vector<vector<long long>> dp(n+2,vector<long long>(n+2,0));
    for(int gap=2;gap<=n+1;++gap)
        for(int i=0;i+gap<=n+1;++i){
            int j=i+gap; long long best=LLONG_MAX;
            for(int k=i+1;k<j;++k)
                best=min(best,dp[i][k]+dp[k][j]);
            dp[i][j]=best+p[j]-p[i];
        }
    cout<<dp[0][n+1]<<'\n';
}
```

12 铺路：按“新升高的层数”计数

题意与样例

有 n 段路，初始高度全 0。一次操作选连续一段，每个高度加 1；求达到目标高度 h[i] 的最少操作次数。

样例：2 1 3 0 2 → 6 次。

分析与状态

从左到右看，每当目标高度比左邻居高，新增的那几层必须在当前位置开始；若下降，已有操作可以在前一位置结束，不需要新增。

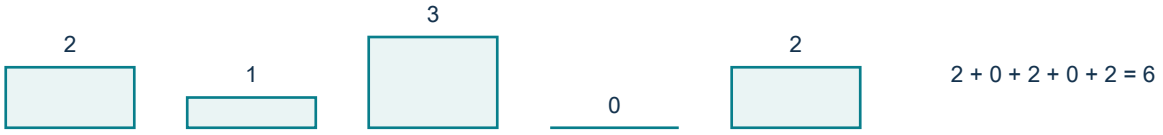
不必建二维 dp。设 h[0]=0，ans 累加各位置正向升高量。

边界、转移与计算顺序

ans = Σ max(0,h[i]−h[i−1])，i 从 1 到 n。可用前缀累加：ans_i=ans_{i−1}+max(0,h[i]−h[i−1])。

复杂度与易错点

复杂度：时间 O(n)，额外空间 O(1)。 常见错误：历史回答里若使用“绝对差求和”“每段高度直接求和”或下降时也加操作的递推，都是错误的；本式经小规模穷举校验。



独立核对：2 1 3 0 2 的上升量为 2、0、2、0、2，总数 6。可以构造 6 次操作：覆盖 1..3 一次，覆盖 1 一次，覆盖 3 两次，覆盖 5 两次。每一处上升量也是不可少的下界，所以 6 恰好最优。

C++14 代码

```
#include <iostream>
#include <algorithm>
using namespace std;
int main(){
    int n;cin>>n;
    long long ans=0,prev=0,h;
    for(int i=1;i<=n;++i){
        cin>>h;
        ans+=max(0LL,h-prev);
        prev=h;
    }
    cout<<ans<<'\n';
}
```

13 Dijkstra：链式前向星与路径回溯

题意与样例

n 个点、m 条非负权有向边，从 s 求到 t 的最短距离及一条最短路径。

样例：边：1→2(2),1→3(5),2→3(1),3→4(2),2→4(6)；s=1,t=4，距离 5，路径 1 2 3 4。

分析与状态

head[u] 指向 u 的第一条边，next[e] 把同一起点的边串起来。优先队列每次取当前距离最小的点，松弛边时记 pre[v]=u。

dist[v] = 当前找到的 s→v 最短候选距离；pre[v] = 该候选路径上 v 的前驱。

边界、转移与计算顺序

dist[s]=0，其余 INF；弹出旧队列项时跳过。所有边权必须 ≥0，且最短距离在 long long 内。算完从 t 顺着 pre 回到 s，再反转；不可达则单独输出。

复杂度与易错点

复杂度：时间 O((n+m)log(n+m))，空间 O(n+m)。 常见错误：负权边不能用本算法；dist[u]+w 溢出；无可达路径却盲目回溯；无向图需要双向加边。

链式前向星图：head[1] → 边(1→3) → 边(1→2) → −1；next 保存同起点上一条边的编号。

C++14 代码

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
#include <functional>
#include <limits>
using namespace std;
struct Edge{int to; long long w; int next;};
int main(){
    int n,m,s,t;cin>>n>>m>>s>>t;
    vector<int> head(n+1,-1),pre(n+1,-1);
    vector<Edge> edges;edges.reserve(m);
    auto add=[&](int u,int v,long long w){
        edges.push_back({v,w,head[u]});head[u]=(int)edges.size()-1;
    };
    for(int i=0;i<m;++i){int u,v;long long w;cin>>u>>v>>w;add(u,v,w);}
    const long long INF=numeric_limits<long long>::max();
    vector<long long> dist(n+1,INF);dist[s]=0;
    using P=pair<long long,int>;
    priority_queue<P,vector<P>,greater<P>> pq;pq.push({0,s});
    while(!pq.empty()){
        P top=pq.top();pq.pop();long long d=top.first;int u=top.second;
        if(d!=dist[u])continue;
        for(int e=head[u];e!=-1;e=edges[e].next){
            int v=edges[e].to;long long w=edges[e].w;
            if(w<=INF-d && d+w<dist[v]){
                dist[v]=d+w;pre[v]=u;pq.push({dist[v],v});
            }
        }
    }
    if(dist[t]==INF){cout<<"unreachable\n";return 0;}
    cout<<dist[t]<<'\n';vector<int> path;
    for(int v=t;v!=-1;v=pre[v])path.push_back(v);
    reverse(path.begin(),path.end());
    for(size_t i=0;i<path.size();++i)
        cout<<path[i]<<(i+1==path.size()?'\n':' ');
}
```

14 猴子桃子：先统一“剩 1 个”的时点

经典条件：每天吃掉“当前桃子的一半再加 1 个”，吃完后必须仍为整数。若第 10 天早上、尚未吃时剩 1 个，则第 1 至第 9 天共发生 9 次“吃”。反推公式是 before = 2×(after+1)。从 1 倒推 9 次得到 1534。

若改成第 10 天吃完以后剩 1 个，就要倒推 10 次，得到
3068。这两个样例不矛盾，条件的时间点不同。历史对话里若把同一个时间点同时对应 1534 和
3068，应视作样例矛盾；原始样例的完整文字未保留，本手册不沿用其未核对的数字。

验算：1534→766→382→190→94→46→22→10→4→1，正好 9 次；3068 再多一次变到 1。

题意、状态与复杂度

输入 n，表示第 n 天早上剩 1 个；输出第一天早上的桃子数。设 x 为下一天早上的数量，倒推 n−1 次 x=2(x+1)。时间 O(n)，空间 O(1)。使用 long long，n 很大时会溢出；本代码假设 1≤n≤60。

C++14 代码

```
#include <iostream>
using namespace std;
int main(){
    int n;cin>>n;
    long long x=1;
    for(int day=n-1;day>=1;--day) x=2*(x+1);
    cout<<x<<'\n';
}
```

附录 A 纸牌均分：一道适合收录的贪心题

题意与样例

n 堆纸牌，总数可被 n 整除；一次可将任意数量纸牌在相邻堆之间搬运，求最少需要跨越多少条相邻边。

样例：9 0 0 31，总数 40，均值 10；答案 3。

分析与状态

从左到右算前缀余额 balance=前 i 堆实际总数−i×均值。balance≠0 表示第 i、i+1 堆之间必须发生转移。

本题不需要 DP；前缀余额是“跨过当前分界线的净流量”。

边界、转移与计算顺序

扫描 i=1..n−1，每次 balance+=a[i]−avg；若 balance≠0，答案加 1。

复杂度与易错点

复杂度：时间 O(n)，空间 O(n)（存输入）；若先知均值，也可 O(1) 额外空间。
常见错误：中间余额为负只是记账，真实搬运可按资源来源调整顺序。

C++14 代码

```
#include <iostream>
#include <vector>
using namespace std;
int main(){
    int n;cin>>n;vector<long long>a(n);
    long long sum=0;
    for(auto &x:a){cin>>x;sum+=x;}
    long long avg=sum/n,balance=0;int ans=0;
    for(int i=0;i<n-1;++i){
        balance+=a[i]-avg;
        if(balance!=0)++ans;
    }
    cout<<ans<<'\n';
}
```

附录 B 交卷前的 8 项检查

1. 先用一句中文重述 dp[i]；“到达 i”与“前 i 个”不能混用。
2. 明确答案位置：dp[0]、dp[n]、还是所有状态的最大值。
3. 用 n=1、全 0、全负数、不可达等边界样例手算。
4. 输入从 0 还是 1 开始存？公式中的每个下标对应哪一个对象？
5. 转移依赖左边还是右边？正序、倒序与 0/1 背包原地更新是否匹配？
6. 求最大值时初值能否为 0？求最小值时 INF 加法是否安全？
7. 整数是否可能超过 int；路径和、方案数与乘法优先考虑 long long。
8. 样例输出应自己算一遍；若样例与文字矛盾，先列清解释再写程序。

> 本手册所有“教学版”题目都写明了自己的规则与输入格式。若你拿到原题完整题面，先比较：能否起跳、是否必须到终点、费用何时支付、空方案是否合法、图是有向还是无向。规则不同，公式也可能不同。

公式速查

最大正方形：1 + min(上,左,左上)；爬楼梯：min(前一阶+其费用,前两阶+其费用)；打家劫舍：max(不偷,偷)；最大子段和：max(从当前重开,接在前段后)；铺路：只加正向高度差。

记忆顺序：先定状态含义 → 列合法前驱/选择 → 设边界 → 决定遍历方向 → 手算样例 → 写代码。